МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное образовательное учреждение высшего образования «Санкт-Петербургский политехнический университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

02.03.01 Математика и компьютерные науки

Отчет по лабораторной работе №1 по дисциплине Дискретная математика

«Реализация операций над мультимножествами, созданными на основе сгенерированного кода Грея»

Выполнил:
Студент группы 5130201/40001 _____ Исаева А.Е.

Проверил:
Преподаватель _____ Востров А.В.

«____»_____ 2025 г.

Санкт-Петербург, 2025

Содержание

Введение 4

1 Математическое описание 5
- 1.1 Бинарный код Грея . . . 5
- 1.2 Множества и мультимножества . . . 5
- 1.3 Операции над множествами и мультимножествами . . . 6
  - 1.3.1 Теоретико-множественные операции . . . 6
  - 1.3.2 Арифметические операции . . . 7

2 Особенности реализации 9
- 2.1 Класс Multiset . . . 9
  - 2.1.1 Поля класса . . . 9
  - 2.1.2 Конструктор пустого мультимножества . . . 9
  - 2.1.3 Конструктор мультимножества из заданного вектора элементов . . . 9
  - 2.1.4 Метод get_multiplicity . . . 10
  - 2.1.5 Метод set_multiplicity . . . 10
  - 2.1.6 Метод get_universe_size . . . 10
  - 2.1.7 Метод total_elements . . . 10
  - 2.1.8 Метод contains . . . 11
  - 2.1.9 Метод is_empty . . . 11
  - 2.1.10 Метод add_element . . . 11
  - 2.1.11 Метод remove_element . . . 11
  - 2.1.12 Метод clear . . . 12
  - 2.1.13 Функция Union . . . 12
  - 2.1.14 Функция Intersection . . . 12
  - 2.1.15 Функция Complement . . . 13
  - 2.1.16 Функция Difference . . . 13
  - 2.1.17 Функция Symmetric_difference . . . 13
  - 2.1.18 Функция Ar_sum . . . 14
  - 2.1.19 Функция Ar_difference . . . 14
  - 2.1.20 Функция Ar_product . . . 14
  - 2.1.21 Функция Ar_division . . . 15
  - 2.1.22 Функция is_subset . . . 15
  - 2.1.23 Функция is_superset . . . 15
  - 2.1.24 Статический метод create_random . . . 16
  - 2.1.25 Метод print . . . 16
  - 2.1.26 Метод print_detailed . . . 16
  - 2.1.27 Метод to_string . . . 17
  - 2.1.28 Метод to_vector . . . 17
  - 2.1.29 Оператор operator== . . . 17
  - 2.1.30 Оператор operator!= . . . 18
- 2.2 Класс Application . . . 18
  - 2.2.1 Поля класса . . . 18
  - 2.2.2 Конструктор по умолчанию . . . 18

2.2.3 Метод configureUniverseMultiplicities 18
2.2.4 Метод getValidInput 19
2.2.5 Метод generateGrayCode 19
2.2.6 Метод displayUniverse() 20
2.2.7 Метод createMultisetsAutomatically 20
2.2.8 Метод createMultisetsManually 21
2.2.9 Метод performOperations 22
2.2.10 Метод run 23
2.3 Функция main 24
2.3.1 Описание функции 24
2.3.2 Структура вызовов 25

3 Результаты работы **26**

Заключение **32**

Список литературы **33**

Введение

В отчёте представлено описание лабораторной работы по дисциплине Дискретная математика. Работа представляет из себя код на языке C++ в объектно-ориентированном стиле, который является реализацией классов мультимножества и пользовательского интерфейса.

Данные классы реализовывают, согласно заданию, программу генерации бинарного кода Грея для заполнения универсума мультмножеств разрядности, заданной пользователем. На основе универсума должны формироваться два мультимножества двумя способами заполнения - вручную и автоматически, по выбору пользователя. Мощности множеств также задает пользователь. В результате на экран выводятся результаты действий над множествами: объединение, пересечение, разность, симметрическая разность, дополнение, арифметические сумма, разность, произведение и деление. Кроме того, требовалось реализовать защиту от некорректного пользовательского ввода и пользовательское меню.

1 Математическое описание

Данный раздел описывает алгоритм построения бинарного кода Грея, понятия множества и мультимножеств и операции над ними, поясняя математическую основу программы.

1.1 Бинарный код Грея

Код Грея – непозиционный код с одним набором символов (0 и 1) для каждого разряда.

Преобразование чисел из позиционного кода в код Грея выполняется по следующему алгоритму: каждый i-й разряд (отсчитывая слева) в коде Грея равен сумме по модулю 2 i-го и $(i-1)$-го разрядов исходного числа в позиционном коде. [1]

То есть, i-й бит кода Грея G_i выражается через биты двоичного кода B_i следующим образом:

$$G_i = B_i \oplus B_{i+1}$$

Ниже приведён алгоритм преобразования из двоичной системы счисления в код Грея, записанный на псевдокоде:

Algorithm 1 Генерация кода Грея итеративным методом

Require: $n \in \mathbb{N}$ – количество битов
Ensure: Список бинарных строк кода Грея длины n

```
gray ← ∅
total ← 2^n                                   ▷ Общее количество кодов
for i ← 0 to total − 1 do
    gray_code ← i ⊕ (i ≫ 1)                   ▷ Побитовое XOR со сдвигом
    gray.append(binary_string(gray_code, n))
end for
return gray
```

1.2 Множества и мультимножества

Первичным понятием теории множеств является понятие самого *множества*.

Множество – это совокупность некоторых (произвольных) объектов, объединённых по какому-либо признаку. Множество обозначается парой фигурных скобок {...}.

Если объект a является элементом множества A, то пишут $a \in A$ (a принадлежит A), запись $a \notin A$ говорит о том, что a не принадлежит A.[2]

Если множество не содержит ни одного элемента, то оно называется *пустым* и обозначается $\varnothing$. [3]

Если мы оперируем подмножествами некоторого большего множества U, то мы называем U *универсальным* множеством (или *универсумом*). То есть для разных задач универсум может быть разным. [4]

Мультимножество – совокупность элементов, в которую элементы входят по несколько раз.

Мультимножество X обозначается следующим способом:

$$X = \{a_1x_1, a_2x_2, ..., a_ix_i\}.$$

Где число a_i обозначает число вхождений элемента x_i и называется *показателем(кратностью* элемента, а число
$m = a_1 + \ldots + a_n$ — *мощностью* мультимножества X. [5]

1.3 Операции над множествами и мультимножествами

Согласно заданию, нужно было реализовать операции для работы с мультимножествами, приведенные в следующих подразделах.

1.3.1 Теоретико-множественные операции

Далее будут описаны теоретико-множественные операции для работы с множествами и мультимножествами. [4]

- **Объединение**

  Объединением двух множеств A и B называется множество, которое состоит из тех элементов, которые принадлежат либо множеству A, либо множеству B, а возможно и обоим сразу.

  Для множеств:

  $$A \cup B = \{x : x \in A \vee x \in B\}.$$

  Для мультимножеств:

  $$A \cup B = \{ax : ax = \max(a_i x_i, a_j x_j)\}, \quad \text{где } a_i x_i \in A,\ a_j x_j \in B.$$

- **Пересечение**

  Пересечением двух множеств A и B называется множество, которое состоит из элементов, которые принадлежат как множеству A, так и множеству B.

  Для множеств:

  $$A \cap B = \{x : x \in A \wedge x \in B\}.$$

  Для мультимножеств:

  $$A \cap B = \{ax : ax = \min(a_i x_i, a_j x_j)\}, \quad \text{где } a_i x_i \in A,\ a_j x_j \in B.$$

- **Разность**

  Разностью множества A и множества B называется множество, которое состоит из всех элементов множества A, которые не принадлежат B.

  Для множеств:

  $$A \setminus B = A \cap \overline{B} = \{x : x \in A \wedge x \notin B\}.$$

  Для мультимножеств:

  $$A \cap B = \{ax : ax = \min(a_i x_i, a_j x_j)\}, \quad \text{где } a_i x_i \in A,\ a_j x_j \in \overline{B}.$$

- **Дополнение**

  Дополнением множества A называется множество, которое состоит из всех элементов множества U(универсума), которые не принадлежат A.

  Для множеств:

  $$\overline{A} = U \setminus A = \{x : x \notin A\}.$$

  Для мультимножеств:

  $$\overline{A} = \{ax : ax = \max(a_i x_i - a_j x_j, 0)\}, \quad \text{где } a_i x_i \in U, a_j x_j \in A.$$

- **Симметрическая разность**

  Симметрической разностью двух множеств A и B называют множество, которое состоит из всех тех и только тех элементов универсального множества, которые либо принадлежат A и не принадлежат B, либо наоборот, принадлежат B, но не A. Грубо говоря, симметрическая разность состоит из элементов, лежащих либо в A, либо в B, но не одновременно.

  Для множеств:

$$A \triangle B = (A \cup B) \setminus (A \cap B) = \{x : (x \in A \wedge x \notin B) \vee (x \in B \wedge x \notin A)\}.$$

  Для мультимножеств:

$$A \triangle B = \{ax : ax = \max\big(\max(a_i x_i, a_j x_j) - \min(a_i x_i, a_j x_j),\ 0\big)\} \quad \text{где } a_i x_i \in A, a_j x_j \in B.$$

1.3.2 Арифметические операции

В данном подразделе описаны арифметические операции для работы с множествами и мультимножествами.

- **Сумма**

  Арифметической суммой множеств A и B называется множество, которое состоит из тех элементов, которые принадлежат либо множеству A, либо множеству B, а возможно и обоим сразу. Кратность каждого элемента равна сумме кратностей соответствующих элементов множеств A и B, однако сумма кратностей не может быть больше, чем кратность данного элемента в универсуме.

  Для множеств:

$$A + B = \{x : x \in A \vee x \in B\}.$$

  Для мультимножеств:

$$A + B = \{ax : ax = \min(a_i x_i + a_j x_j, u_k x_k)\}, \quad \text{где } a_i x_i \in A, a_j x_j \in B, u_k x_k \in U.$$

- **Разность**

  Арифметической разностью множеств A и B называется множество, которое состоит из элементов, кратность которых получается путем вычитания из кратности элемента A кратности элемента B. Кратность эта, однако, не может быть ниже нуля.

  Для множеств:

$$A - B = \{x : x \in A \wedge x \notin B\}.$$

  Для мультимножеств:

$$A - B = \{ax : ax = \max(a_i x_i - a_j x_j, 0)\}, \quad \text{где } a_i x_i \in A, a_j x_j \in B.$$

- **Произведение**

  Арифметическим произведением двух множеств A и B называется множество, которое состоит из элементов, которые принадлежат как множеству A, так и множеству B. Кратность каждого элемента рассчитывается как произведение кратностей соответствующих элементов A и B и не может быть больше, чем кратность данного элемента в универсуме.

Для множеств:

$$A * B = \{x : x \in A \wedge x \in B\}.$$

Для мультимножеств:

$$A * B = \{ax : ax = \min(a_i x_i * a_j x_j, u_k x_k)\}, \quad \text{где } a_i x_i \in A, a_j x_j \in B, u_k x_k \in U.$$

- **Деление**

  Арифметическим делением двух множеств A и B называется операция, после выполнения которой получается множество, которое состоит из элементов, кратность которых задается как частное кратностей соответствующих элементов A и B. Если кратность элемента B равна 0, результат деления равен 0.

  Для мультимножеств:

$$A : B = \left\{ ax : ax = \left\lfloor \frac{a_i x_i}{a_j x_j} \right\rfloor \vee a_j x_j = 0 => 0 \right\}, \quad \text{где } a_i x_i \in A, a_j x_j \in B,$$

  а скобки $\lfloor\ \rfloor$ означают целочисленное деление.

2 Особенности реализации

Программа состоит из двух классов и функции main, которые являются реализацией консольного приложения для работы с мультимножествами.

После начала работы программы и входе в функцию main, с помощью экземпляра класса Application запускается пользовательское меню, которое, в свою очередь, выполняя действия, выбранные пользователем, создает и работает с экземплярами класса Multiset, который помогает хранить мультимножества и проводить с ними операции.

Рассмотрим реализацию классов Application и Multiset.

2.1 Класс Multiset

2.1.1 Поля класса

Поля класса использовались Multiset используются для хранения мультимножеств и других величин, которые использовались в реализации Multiset:

- std::vector<int> multiplicities - целочисленный динамический массив для хранения мультимножества, индекс - элемент, значение - кратность данного элемента.
- int universe_size - размер универсума.
- std::vector<int> universe - динамический целочисленный массив для хранения максимальных кратностей универсума.

2.1.2 Конструктор пустого мультимножества

Рассмотрим конструкторы, реализованные в классе Multiset.

```
1  Multiset::Multiset(int size, const std::vector<int>& univ)
2  : universe_size(size), universe(univ), multiplicities(size, 0) {
3  }
```

Листинг 1: Конструктор пустого мультимножества

Описание: Создает пустое мультимножество заданного размера.
Вход: `size` - размер универсума, `univ` - вектор максимальных кратностей.
Выход: Созданный объект пустого мультимножества.
Принцип работы: Инициализирует векторы размером `size` и устанавливает все кратности в 0.

2.1.3 Конструктор мультимножества из заданного вектора элементов

```
1  Multiset::Multiset(const std::vector<int>& elements, int size, const std::vector<int>& univ)
2  : universe_size(size), universe(univ), multiplicities(size, 0) {
3    for (int elem : elements) {
4      if (elem >= 0 && elem < universe_size) {
5        add_element(elem);
6      }
7    }
8  }
```

Листинг 2: Конструктор из списка элементов

Описание: Создает мультимножество из вектора элементов.
Вход: `elements` - вектор элементов, `size` - размер универсума, `univ` - вектор максимальных кратностей.
Выход: Созданный объект мультимножества.
Принцип работы: Для каждого элемента из входного вектора вызывает метод `add_element`.

2.1.4 Метод get_multiplicity

Рассмотрим методы доступа к данным.

```
int Multiset::get_multiplicity(int element) const {
  if (element < 0 || element >= universe_size) return 0;
  return multiplicities[element];
}
```

Листинг 3: Получение кратности элемента

Описание: Возвращает кратность указанного элемента.
Вход: `element` - индекс элемента в универсуме.
Выход: Целое число - кратность элемента.
Принцип работы: Проверяет границы индекса и возвращает значение из вектора кратностей.

2.1.5 Метод set_multiplicity

```
void Multiset::set_multiplicity(int element, int count) {
  if (element >= 0 && element < universe_size && count >= 0) {
    multiplicities[element] = std::min(count, universe[element]);
  }
}
```

Листинг 4: Установка кратности элемента

Описание: Устанавливает кратность элемента с ограничением по максимуму.
Вход: `element` - индекс элемента, `count` - новая кратность.
Выход: Мультимножество с обновленной кратностью элемента.
Принцип работы: Не позволяет установить кратность больше максимальной для данного элемента.

2.1.6 Метод get_universe_size

```
int Multiset::get_universe_size() const {
  return universe_size;
}
```

Листинг 5: Получение размера универсума

Описание: Возвращает размер универсума.
Вход: Объект класса `Multiset`.
Выход: Целое число - размер универсума.
Принцип работы: Просто возвращает значение поля `universe_size`.

2.1.7 Метод total_elements

```
int Multiset::total_elements() const {
  int total = 0;
  for (int count : multiplicities) {
    total += count;
  }
  return total;
}
```

Листинг 6: Подсчет общего количества элементов

Описание: Возвращает общее количество элементов (сумму всех кратностей).
Вход: Объект класса `Multiset`.
Выход: Целое число - сумма всех кратностей.
Принцип работы: Суммирует все значения в векторе кратностей.

2.1.8 Метод contains

```
bool Multiset::contains(int element) const {
  return get_multiplicity(element) > 0;
}
```

Листинг 7: Проверка наличия элемента

Описание: Проверяет, содержится ли элемент в мультимножестве.
Вход: `element` - индекс элемента.
Выход: `true`, если элемент присутствует, иначе `false`.
Принцип работы: Проверяет, что кратность элемента больше нуля.
Ссылки: Использует `get_multiplicity()` (3).

2.1.9 Метод is_empty

```
bool Multiset::is_empty() const {
  for (int count : multiplicities) {
    if (count > 0) return false;
  }
  return true;
}
```

Листинг 8: Проверка пустоты

Описание: Проверяет, является ли мультимножество пустым.
Вход: Объект класса `Multiset`.
Выход: `true`, если мультимножество пустое, иначе `false`.
Принцип работы: Ищет хотя бы один ненулевой элемент в векторе кратностей.

2.1.10 Метод add_element

Рассмотрим методы, модифицирующие мультимножество.

```
void Multiset::add_element(int element, int count) {
  if (element >= 0 && element < universe_size && count > 0) {
    if (multiplicities[element] + count <= universe[element]) {
      multiplicities[element] += count;
    }
    else {
      cout << "ОШИБКА: Превышена максимальная кратность ("
      << universe[element] << ") для элемента " << element << endl;
    }
  }
}
```

Листинг 9: Добавление элемента

Описание: Добавляет элемент в мультимножество.
Вход: `element` - индекс элемента, `count` - количество добавляемых экземпляров.
Выход: Мультимножество с обновленной(увеличенной) кратностью добавляемого элемента.
Принцип работы: Проверяет, не превышает ли новая кратность максимально допустимую.
Ссылки: Используется в конструкторе из вектора (2).

2.1.11 Метод remove_element

```
void Multiset::remove_element(int element, int count) {
  if (element >= 0 && element < universe_size && count > 0) {
    multiplicities[element] = std::max(0, multiplicities[element] - count);
```

```
    }
  }
```

Листинг 10: Удаление элемента

Описание: Уменьшает кратность элемента.
Вход: `element` - индекс элемента, `count` - количество удаляемых экземпляров.
Выход: Мультимножество с обновленной(уменьшенной) кратностью удаляемого элемента.
Принцип работы: Не позволяет кратности стать отрицательной.

2.1.12 Метод clear

```
void Multiset::clear() {
  std::fill(multiplicities.begin(), multiplicities.end(), 0);
}
```

Листинг 11: Очистка мультимножества

Описание: Очищает мультимножество (устанавливает все кратности в 0).
Вход: Объект класса `Multiset`.
Выход: Мультимножество с кратностями, равными 0.
Принцип работы: Использует стандартный алгоритм `std::fill`.

2.1.13 Функция Union

Далее будут рассмотрены функции, реализующие теоретико-множественные операции для мультимножеств.

```
Multiset Multiset::Union(const Multiset& other) const {
  if (universe_size == 0) return Multiset(0, universe);

  Multiset result(universe_size, universe);
  for (int i = 0; i < universe_size; ++i) {
    result.multiplicities[i] = std::max(multiplicities[i], other.multiplicities[i]);
  }
  return result;
}
```

Листинг 12: Объединение мультимножеств

Описание: Возвращает объединение двух мультимножеств.
Вход: `other` - второе мультимножество.
Выход: Новый объект `Multiset` - результат объединения.
Принцип работы: Для каждого элемента берется максимальная кратность из двух множеств.

2.1.14 Функция Intersection

```
Multiset Multiset::Intersection(const Multiset& other) const {
  if (universe_size == 0) return Multiset(0, universe);

  Multiset result(universe_size, universe);
  for (int i = 0; i < universe_size; ++i) {
    result.multiplicities[i] = std::min(multiplicities[i], other.multiplicities[i]);
  }
  return result;
}
```

Листинг 13: Пересечение мультимножеств

Описание: Возвращает пересечение двух мультимножеств.
Вход: `other` - второе мультимножество.
Выход: Новый объект `Multiset` - результат пересечения.
Принцип работы: Для каждого элемента берется минимальная кратность из двух множеств.

2.1.15 Функция Complement

```
Multiset Multiset::Complement() const {
  if (universe_size == 0) return Multiset(0, universe);

  // !A = U - A
  Multiset result(universe_size, universe);
  for (int i = 0; i < universe_size; ++i) {
    result.multiplicities[i] = universe[i] - multiplicities[i];
  }
  return result;
}
```

Листинг 14: Дополнение мультимножества

Описание: Возвращает дополнение мультимножества до универсума.
Вход: Универсум `universe` и объект класса `Multiset`.
Выход: Новый объект `Multiset` - дополнение.
Принцип работы: Для каждого элемента вычисляет разницу между максимальной и текущей кратностью.

2.1.16 Функция Difference

```
Multiset Multiset::Difference(const Multiset& other) const {
  if (universe_size == 0) return Multiset(0, universe);

  // A \ B = A n (!B)
  Multiset notB = other.Complement();
  return this->Intersection(notB);
}
```

Листинг 15: Разность мультимножеств

Описание: Разность мультимножеств (A \B).
Вход: `other` - вычитаемое мультимножество.
Выход: Новый объект `Multiset` - результат разности.
Принцип работы: Реализована через пересечение с дополнением.
Ссылки: Использует `Complement()` (14) и `Intersection()` (13).

2.1.17 Функция Symmetric_difference

```
Multiset Multiset::Symmetric_difference(const Multiset& other) const {
  if (universe_size == 0) return Multiset(0, universe);

  // A (+) B = (A U B) \ (A n B)
  Multiset union_set = this->Union(other);
  Multiset intersection_set = this->Intersection(other);
  return union_set.Difference(intersection_set);
}
```

Листинг 16: Симметрическая разность

Описание: Симметрическая разность мультимножеств.
Вход: `other` - второе мультимножество.
Выход: Новый объект `Multiset` - симметрическая разность.

Принцип работы: Вычисляется как разность объединения и пересечения.
Ссылки: Использует `Union()` (12), `Intersection()` (13), `Difference()` (15).

2.1.18 Функция Ar_sum

Затем рассмотрим арифметические операции над мультимножествами.

```
Multiset Multiset::Ar_sum(const Multiset& other) const {
  if (universe_size == 0) return Multiset(0, universe);

  Multiset result(universe_size, universe);
  for (int i = 0; i < universe_size; ++i) {
    int sum = multiplicities[i] + other.multiplicities[i];
    result.multiplicities[i] = std::min(sum, universe[i]);
  }
  return result;
}
```

Листинг 17: Арифметическая сумма

Описание: Арифметическая сумма кратностей.
Вход: `other` - второе мультимножество.
Выход: Новый объект `Multiset` - арифметическая сумма.
Принцип работы: Складывает кратности и ограничивает результат максимальным значением.

2.1.19 Функция Ar_difference

```
Multiset Multiset::Ar_difference(const Multiset& other) const {
  if (universe_size == 0) return Multiset(0, universe);

  Multiset result(universe_size, universe);
  for (int i = 0; i < universe_size; ++i) {
    result.multiplicities[i] = std::max(0, multiplicities[i] - other.multiplicities[i]);
  }
  return result;
}
```

Листинг 18: Арифметическая разность

Описание: Арифметическая разность кратностей.
Вход: `other` - вычитаемое мультимножество.
Выход: Новый объект `Multiset` - арифметическая разность.
Принцип работы: Вычитает кратности, результат не может быть отрицательным.

2.1.20 Функция Ar_product

```
Multiset Multiset::Ar_product(const Multiset& other) const {
  if (universe_size == 0) return Multiset(0, universe);

  Multiset result(universe_size, universe);
  for (int i = 0; i < universe_size; ++i) {
    int product = multiplicities[i] * other.multiplicities[i];
    result.multiplicities[i] = std::min(product, universe[i]);
  }
  return result;
}
```

Листинг 19: Арифметическое произведение

Описание: Арифметическое произведение кратностей.
Вход: `other` - второе мультимножество.
Выход: Новый объект `Multiset` - арифметическое произведение.
Принцип работы: Перемножает кратности и ограничивает результат.

2.1.21 Функция Ar_division

```
Multiset Multiset::Ar_division(const Multiset& other) const {
  if (universe_size == 0) return Multiset(0, universe);

  Multiset result(universe_size, universe);
  for (int i = 0; i < universe_size; ++i) {
    if (other.multiplicities[i] != 0) {
      result.multiplicities[i] = multiplicities[i] / other.multiplicities[i];
    } else {
      result.multiplicities[i] = 0;
    }
  }
  return result;
}
```

Листинг 20: Арифметическое деление

Описание: Целочисленное деление кратностей.
Вход: `other` - делитель (мультимножество).
Выход: Новый объект `Multiset` - результат целочисленного деления.
Принцип работы: Выполняет целочисленное деление с проверкой деления на ноль.

2.1.22 Функция is_subset

Рассмотрим функции, реализующие проверки свойств мультимножеств.

```
bool Multiset::is_subset(const Multiset& other) const {
  if (universe_size == 0) return true;

  for (int i = 0; i < universe_size; ++i) {
    if (multiplicities[i] > other.multiplicities[i]) {
      return false;
    }
  }
  return true;
}
```

Листинг 21: Проверка подмножества

Описание: Проверяет, является ли текущее множество подмножеством другого.
Вход: `other` - потенциальное надмножество.
Выход: `true`, если текущее множество является подмножеством `other`, иначе `false`.
Принцип работы: Для всех элементов проверяет, что кратность в текущем множестве не больше, чем в другом.

2.1.23 Функция is_superset

```
bool Multiset::is_superset(const Multiset& other) const {
  if (universe_size == 0) return other.is_empty();
  return other.is_subset(*this);
}
```

Листинг 22: Проверка надмножества

Описание: Проверяет, является ли текущее множество надмножеством другого.
Вход: `other` - потенциальное подмножество.
Выход: `true`, если текущее множество является надмножеством `other`, иначе `false`.
Принцип работы: Использует проверку подмножества в обратном порядке.
Ссылки: Использует `is_subset()` (21).

2.1.24 Статический метод create_random

```
Multiset Multiset::create_random(int total_elements, int universe_size,
const std::vector<int>& universe) {
  Multiset result(universe_size, universe);
  std::random_device rd;
  std::mt19937 gen(rd());
  std::uniform_int_distribution<> dist(0, universe_size - 1);

  int attempts = 0;
  int max_attempts = total_elements * 10;

  for (int i = 0; i < total_elements && attempts < max_attempts; ++i) {
    int elem = dist(gen);

    if (result.get_multiplicity(elem) < universe[elem]) {
      result.add_element(elem);
    }
    else {
      i--;
    }
    attempts++;
  }

  if (attempts >= max_attempts) {
    cout << "ОШИБКА: Не удалось создать множество..." << endl;
  }

  return result;
}
```

Листинг 23: Создание случайного мультимножества

Описание: Создает случайное мультимножество.
Вход: `total_elements` - общее количество элементов, `universe_size` - размер универсума, `universe` - вектор максимальных кратностей.
Выход: Новый объект `Multiset` - случайное мультимножество.
Принцип работы: Генерирует случайные элементы с учетом ограничений кратности.

2.1.25 Метод print

Далее будут приведены методы, помогающие делать вывод мультимножеств.

```
void Multiset::print() const {
  std::cout << to_string() << std::endl;
}
```

Листинг 24: Вывод мультимножества

Описание: Выводит мультимножество в стандартный поток.
Вход: Объект класса `Multiset`.
Выход: Выведенное в терминал текстовое представление мультимножества.
Принцип работы: Использует метод `to_string()`.
Ссылки: Использует `to_string()` (26).

2.1.26 Метод print_detailed

```
void Multiset::print_detailed() const {
  std::cout << "{ ";
    bool first = true;
    for (int i = 0; i < universe_size; ++i) {
      if (multiplicities[i] > 0) {
        if (!first) std::cout << ", ";
        std::cout << i;
        if (multiplicities[i] > 0) {
          std::cout << "(" << multiplicities[i] << ")";
        }
```

```
            first = false;
          }
        }
        std::cout << " }" << std::endl;
    }
```

Листинг 25: Детализированный вывод

Описание: Выводит мультимножество с указанием кратностей.
Вход: Объект класса `Multiset`.
Выход: Выведенное в терминал мультимножество с указанием кратностей.
Принцип работы: Показывает только ненулевые элементы с их кратностями в скобках.

2.1.27 Метод to_string

```
    std::string Multiset::to_string() const {
      std::stringstream ss;
      ss << "{ ";
        bool first = true;
        for (int i = 0; i < universe_size; ++i) {
          for (int j = 0; j < multiplicities[i]; ++j) {
            if (!first) ss << ", ";
            ss << i;
            first = false;
          }
        }
        ss << " }";
      return ss.str();
    }
```

Листинг 26: Преобразование в строку

Описание: Преобразует мультимножество в строковое представление.
Вход: Объект класса `Multiset`.
Выход: Строка `std::string` - текстовое представление мультимножества.
Принцип работы: Каждый элемент повторяется столько раз, какова его кратность.

2.1.28 Метод to_vector

```
    std::vector<int> Multiset::to_vector() const {
      std::vector<int> result;
      for (int i = 0; i < universe_size; ++i) {
        for (int j = 0; j < multiplicities[i]; ++j) {
          result.push_back(i);
        }
      }
      return result;
    }
```

Листинг 27: Преобразование в вектор

Описание: Преобразует мультимножество в вектор элементов.
Вход: Объект класса `Multiset`.
Выход: Вектор `std::vector<int>` - элементы мультимножества.
Принцип работы: Аналогично `to_string()`, но возвращает вектор вместо строки.

2.1.29 Оператор operator==

Рассмотрим перегрузку операторов сравнения.

```
    bool Multiset::operator==(const Multiset& other) const {
      return multiplicities == other.multiplicities;
    }
```

Листинг 28: Оператор равенства

Описание: Проверка равенства мультимножеств.
Вход: `other` - второе мультимножество.
Выход: `true`, если мультимножества равны, иначе `false`.
Принцип работы: Сравнивает векторы кратностей поэлементно.

2.1.30 Оператор operator!=

```
bool Multiset::operator!=(const Multiset& other) const {
  return !(*this == other);
}
```

Листинг 29: Оператор неравенства

Описание: Проверка неравенства мультимножеств.
Вход: `other` - второе мультимножество.
Выход: `true`, если мультимножества не равны, иначе `false`.
Принцип работы: Отрицание оператора равенства.
Ссылки: Использует оператор == (28).

2.2 Класс Application

2.2.1 Поля класса

- std::vector<int> universe - вектор, содержащий коды Грея (универсум)
- Multiset A, B - два мультимножества для выполнения операций
- int universe_size - количество элементов в универсуме (размер вектора universe)
- std::vector<int> universe_multiplicities - максимальные кратности для каждого элемента универсума

2.2.2 Конструктор по умолчанию

```
Application::Application() : universe_size(0), A(0, vector<int>()), B(0, vector<int>()) {}
```

Листинг 30: Конструктор класса Application

Описание: Конструктор по умолчанию.
Вход: Поля класса `Application`: `universe_size`, `A`, `B`.
Выход: Созданный объект класса `Application`.
Принцип работы: Инициализирует все поля нулевыми значениями. Создает пустые мультимножества A и B.

2.2.3 Метод configureUniverseMultiplicities

```
void Application::configureUniverseMultiplicities() {
  cout << "\n=== НАСТРОЙКА КРАТНОСТЕЙ УНИВЕРСУМА ===" << endl;

  if (universe_size == 0) {
    cout << "Универсум пустой - кратности не требуются." << endl;
    return;
  }

  universe_multiplicities.clear();
  universe_multiplicities.resize(universe_size);

  cout << "Заполнение кратностей для " << universe_size << " элементов:" << endl;
  cout << "1. Случайные значения (1-10)" << endl;
```

```
    cout << "2. Вручную" << endl;

    int choice = getValidInput("Ваш выбор: ", 1, 2);

    if (choice == 1) {
      random_device rd;
      mt19937 gen(rd());
      uniform_int_distribution<> dist(1, 10);

      for (int i = 0; i < universe_size; ++i) {
        universe_multiplicities[i] = dist(gen);
      }
      cout << "Кратности установлены случайным образом!" << endl;
    }
    else {
      for (int i = 0; i < universe_size; ++i) {
        universe_multiplicities[i] = getValidInput(
        "Кратность для элемента " + to_string(i) + ": ", 1, 100000);
      }
    }

    A = Multiset(universe_size, universe_multiplicities);
    B = Multiset(universe_size, universe_multiplicities);
  }
```

Листинг 31: Настройка кратностей универсума

Описание: Настраивает максимальные кратности элементов универсума.
Вход: Количество элементов `universe_size`.
Выход: Универсум с максимальными кратностями `universe_multiplicities` и пустые мультимножества A и B.
Принцип работы: Предлагает пользователю выбрать способ заполнения кратностей: случайный или ручной. После установки кратностей создает пустые мультимножества A и B.
Ссылки: Использует `getValidInput()` (32).

2.2.4 Метод getValidInput

```
  int Application::getValidInput(const string& prompt, int min_val, int max_val) {
    int value;
    while (true) {
      cout << prompt;
      cin >> value;
      if (cin.fail() || value < min_val || value > max_val) {
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        cout << "Ошибка! Введите число от " << min_val << " до " << max_val << ": ";
      }
      else {
        break;
      }
    }
    return value;
  }
```

Листинг 32: Валидация пользовательского ввода

Описание: Получает и валидирует пользовательский ввод.
Вход: `prompt` - строка приглашения, `min_val` - минимальное допустимое значение, `max_val` - максимальное допустимое значение.
Выход: Целое число - корректное значение, введенное пользователем.
Принцип работы: В цикле запрашивает ввод до тех пор, пока не будет получено корректное значение в заданном диапазоне. Обрабатывает ошибки ввода.

2.2.5 Метод generateGrayCode

```
vector<int> Application::generateGrayCode(int bits) {
  vector<int> gray_code;
  if (bits == 0) {
    cout << "При заданной разрядности(0), универсум - пустое множество." << endl;
    return gray_code; // пустой вектор
  }

  unsigned long n = 1ULL << bits;
  for (unsigned long i = 0; i < n; i++) {
    gray_code.push_back(static_cast<int>(i ^ (i >> 1)));
  }
  return gray_code;
}
```

Листинг 33: Генерация кода Грея

Описание: Генерирует последовательность кодов Грея заданной разрядности.
Вход: `bits` - разрядность кода Грея.
Выход: Вектор `std::vector<int>` - коды Грея.
Принцип работы: Использует алгоритм преобразования через XOR: $G(i) = i \oplus (i >> 1)$. Генерирует 2^{bits} кодов.

2.2.6 Метод displayUniverse()

```
void Application::displayUniverse() const {
  cout << "\n=== УНИВЕРСУМ (код Грея) ===" << endl;

  if (universe.empty()) {
    cout << "Пустой универсум (разрядность 0)" << endl;
  }
  else {
    int bits = static_cast<int>(log2(universe.size()));

    for (size_t i = 0; i < universe.size(); ++i) {
      string binary_str = "";
      int num = universe[i];

      for (int j = bits - 1; j >= 0; --j) {
        binary_str += (num & (1 << j)) ? '1' : '0';
      }

      cout << binary_str << " : " << universe[i] << " (макс. кратность: " <<
  universe_multiplicities[i] << ")" << endl;
    }
  }
  int multies = 0;
  for (int i = 0; i < universe.size(); i++) {
    multies += universe_multiplicities[i];
  }
  cout << "Мощность универсума(сумма макс. кратностей): " << multies << endl;
}
```

Листинг 34: Отображение универсума

Описание: Отображает универсум в удобочитаемом формате.
Вход: Универсум `universe` и его кратности `universe_multiplicities`.
Выход: Выведенные в терминал универсум и его мощность.
Принцип работы: Для каждого кода Грея выводит двоичное представление, десятичное значение и максимальную кратность. В конце выводит суммарную мощность универсума.

2.2.7 Метод createMultisetsAutomatically

```
void Application::createMultisetsAutomatically() {
  int total_max = accumulate(universe_multiplicities.begin(), universe_multiplicities.end(),
   0);
  int powerA = getValidInput("Введите мощность множества A: ", 0, total_max);
```

```
    int powerB = getValidInput("Введите мощность множества B: ", 0, total_max);

    A = Multiset::create_random(powerA, universe_size, universe_multiplicities);
    B = Multiset::create_random(powerB, universe_size, universe_multiplicities);

    cout << "Мультимножества созданы автоматически!" << endl;
  }
```

Листинг 35: Автоматическое создание мультимножеств

Описание: Создает мультимножества A и B автоматически.
Вход: Универсум `universe` и его кратности `universe_multiplicities`.
Выход: Созданные автоматически мультимножества A и B.
Принцип работы: Запрашивает мощности множеств и создает их с помощью статического метода `Multiset::create_random`.
Ссылки: Использует `getValidInput()` (32) и `Multiset::create_random` (23).

2.2.8 Метод createMultisetsManually

```
  void Application::createMultisetsManually() {
    cout << "\n--- Создание множества A ---" << endl;
    int total_max = accumulate(universe_multiplicities.begin(), universe_multiplicities.end(),
     0);
    int powerA = getValidInput("Введите количество элементов для A: ", 0, total_max);

    vector<int> elementsA;
    if (powerA > 0) {
      cout << "Введите " << powerA << " элементов из универсума [0-" << universe_size - 1 << "
    ]:" << endl;
      for (int i = 0; i < powerA; ++i) {
        int elem = getValidInput("Элемент: ", 0, universe_size - 1);

        int current_count = count(elementsA.begin(), elementsA.end(), elem);
        if (current_count >= universe_multiplicities[elem]) {
          cout << "Предупреждение: элемент " << elem << " уже достиг максимальной кратности ("
          << universe_multiplicities[elem] << "). Введите другой элемент." << endl;
          i--;
          continue;
        }
        elementsA.push_back(elem);
      }
    }
    A = Multiset(elementsA, universe_size, universe_multiplicities);

    cout << "\n--- Создание множества B ---" << endl;
    int powerB = getValidInput("Введите количество элементов для B: ", 0, total_max);

    vector<int> elementsB;
    if (powerB > 0) {
      cout << "Введите " << powerB << " элементов из универсума [0-" << universe_size - 1 << "
    ]:" << endl;
      for (int i = 0; i < powerB; ++i) {
        int elem = getValidInput("Элемент: ", 0, universe_size - 1);

        int current_count = count(elementsB.begin(), elementsB.end(), elem);
        if (current_count >= universe_multiplicities[elem]) {
          cout << "Предупреждение: элемент " << elem << " уже достиг максимальной кратности ("
          << universe_multiplicities[elem] << "). Введите другой элемента." << endl;
          i--;
          continue;
        }
        elementsB.push_back(elem);
      }
    }
    B = Multiset(elementsB, universe_size, universe_multiplicities);

    cout << "Мультимножества созданы вручную!" << endl;
  }
```

Листинг 36: Ручное создание мультимножеств

Описание: Создает мультимножества A и B вручную.
Вход: Универсум `universe` и его кратности `universe_multiplicities`.
Выход: Созданные вручную множества A и B.
Принцип работы: Поэлементно запрашивает элементы для каждого множества с проверкой ограничений кратности.
Ссылки: Использует `getValidInput()` (32) и конструктор `Multiset` (2).

2.2.9 Метод performOperations

```
void Application::performOperations() const {
    cout << "\n=== РЕЗУЛЬТАТЫ ОПЕРАЦИЙ ===" << endl;

    cout << "\nМультимножество A: ";
    A.print_detailed();
    cout << "Мультимножество B: ";
    B.print_detailed();

    cout << "\nОТНОШЕНИЯ МЕЖДУ МНОЖЕСТВАМИ:" << endl;
    cout << "- A является подмножеством B: " << (A.is_subset(B) ? "ДА" : "НЕТ") << endl;
    cout << "- B является подмножеством A: " << (B.is_subset(A) ? "ДА" : "НЕТ") << endl;
    cout << "- A является надмножеством B: " << (A.is_superset(B) ? "ДА" : "НЕТ") << endl;
    cout << "- B является надмножеством A: " << (B.is_superset(A) ? "ДА" : "НЕТ") << endl;
    cout << "- Множества равны: " << (A == B ? "ДА" : "НЕТ") << endl;
    cout << "- Множества не равны: " << (A != B ? "ДА" : "НЕТ") << endl;

    cout << "\nТЕОРЕТИКО-МНОЖЕСТВЕННЫЕ ОПЕРАЦИИ:" << endl;
    cout << "- Объединение (A U B): ";
    A.Union(B).print_detailed();

    cout << "- Пересечение (A n B): ";
    A.Intersection(B).print_detailed();

    cout << "- Разность (A \\ B): ";
    A.Difference(B).print_detailed();

    cout << "- Разность (B \\ A): ";
    B.Difference(A).print_detailed();

    cout << "- Симметрическая разность (A (+) B): ";
    A.Symmetric_difference(B).print_detailed();

    cout << "- Дополнение A (!A): ";
    A.Complement().print_detailed();

    cout << "- Дополнение B (!B): ";
    B.Complement().print_detailed();

    cout << "\nАРИФМЕТИЧЕСКИЕ ОПЕРАЦИИ:" << endl;
    cout << "- Сумма (A + B): ";
    A.Ar_sum(B).print_detailed();

    cout << "- Разность (A - B): ";
    A.Ar_difference(B).print_detailed();

    cout << "- Произведение (A * B): ";
    A.Ar_product(B).print_detailed();

    cout << "- Деление (A / B): ";
    A.Ar_division(B).print_detailed();

    cout << "- Деление (B / A): ";
    B.Ar_division(A).print_detailed();
}
```

Листинг 37: Выполнение операций над мультимножествами

Описание: Выполняет все операции над мультимножествами A и B.
Вход: Мультимножества A и B.
Выход: Вывод результатов операций над множествами.

Принцип работы: Последовательно выполняет и выводит результаты всех теоретико-множественных и арифметических операций.
Ссылки: Использует методы класса Multiset: `print_detailed()` (25), `is_subset()` (21), `Union()` (12), `Intersection()` (13), `Difference()` (15), `Symmetric_difference()` (16), `Complemen` (14), `Ar_sum()` (17), `Ar_difference()` (18), `Ar_product()` (19), `Ar_division()` (20).

2.2.10 Метод run

```
void Application::run() {
  setlocale(LC_ALL, "Russian");

  while (true) {
    cout << "\n=== ГЛАВНОЕ МЕНЮ ===" << endl;
    cout << "1. Сгенерировать универсум" << endl;
    cout << "2. Настроить кратности универсума" << endl;
    cout << "3. Создать мультимножества" << endl;
    cout << "4. Выполнить операции" << endl;
    cout << "5. Показать текущие данные" << endl;
    cout << "6. Выход" << endl;

    int choice = getValidInput("Выберите действие: ", 1, 6);

    switch (choice) {
      case 1: {
        int bits = getValidInput("Введите разрядность кода Грея (0-11): ", 0, 11);
        universe = generateGrayCode(bits);
        universe_size = universe.size();
        configureUniverseMultiplicities();
        displayUniverse();
        break;
      }
      case 2: {
        if (universe_size == 0) {
          cout << "Универсум пустой - кратности не требуются." << endl;
          break;
        }
        configureUniverseMultiplicities();
        break;
      }
      case 3: {
        if (universe_size == 0) {
          cout << "Универсум пустой - создание пустых множеств A и B." << endl;
          A = Multiset(0, vector<int>());
          B = Multiset(0, vector<int>());
          break;
        }
        cout << "\nСпособ заполнения:" << endl;
        cout << "1. Автоматически" << endl;
        cout << "2. Вручную" << endl;
        int method = getValidInput("Ваш выбор: ", 1, 2);

        if (method == 1) {
          createMultisetsAutomatically();
        }
        else {
          createMultisetsManually();
        }
        break;
      }
      case 4: {
        performOperations();
        break;
      }
      case 5: {
        if (!universe.empty()) {
          displayUniverse();
        }
        if (!A.is_empty()) {
          cout << "Мультимножество A: ";
          A.print_detailed();
        }
        else {
```

```
65              cout << "Мультимножество A: { } (пустое)" << endl;
66            }
67            if (!B.is_empty()) {
68              cout << "Мультимножество B: ";
69              B.print_detailed();
70            }
71            else {
72              cout << "Мультимножество B: { } (пустое)" << endl;
73            }
74            break;
75          }
76          case 6:
77          cout << "Выход из программы." << endl;
78          return;
79        }
80      }
81    }
```

Листинг 38: Главный цикл приложения

Описание: Главный цикл выполнения программы.
Вход: Ввод пользователя.
Выход: Пользовательское меню.
Принцип работы: Реализует интерактивное меню с 6 пунктами, обрабатывает выбор пользователя и вызывает соответствующие методы.
Ссылки: Использует все основные методы класса: `getValidInput()` (32), `generateGrayCode()` (33), `configureUniverseMultiplicities()` (31), `displayUniverse()` (34), `createMultisetsAuto` (35), `createMultisetsManually()` (36), `performOperations()` (37).

2.3 Функция main

2.3.1 Описание функции

```
1   #include "application.h"
2
3   int main() {
4     Application app;
5     app.run();
6     return 0;
7   }
```

Листинг 39: Главная функция программы

Описание: Точка входа в программу.
Вход: Объект класса `Application`.
Выход: Целое число - код возврата (0 при успешном завершении).
Принцип работы:

- Создает экземпляр класса `Application`
- Вызывает метод `run()` для запуска главного цикла программы
- Возвращает 0 при успешном завершении работы

Связь с другими компонентами:

- Подключает заголовочный файл `application.h`
- Создает объект класса `Application` (конструктор 30)
- Вызывает метод `run()` (38) для запуска программы

2.3.2 Структура вызовов

```
main()
1. Application::Application()    (конструктор)
2. Application::run()           (главный цикл)
 2.1. getValidInput()        (валидация ввода)
 2.2. generateGrayCode()     (генерация Грея)
 2.3. configureUniverseMultiplicities() (настройка)
 2.4. displayUniverse()      (отображение)
 2.5. createMultisetsAutomatically() или
 2.6. createMultisetsManually() (создание)
 2.7. performOperations()    (выполнение операций)
```

3 Результаты работы

В данном разделе будут приведены результаты работы программы: пользовательское меню и его возможности, проверки защиты от некорректного ввода.

На рис.1 представлено меню, встречающее пользователя при запуске программы.

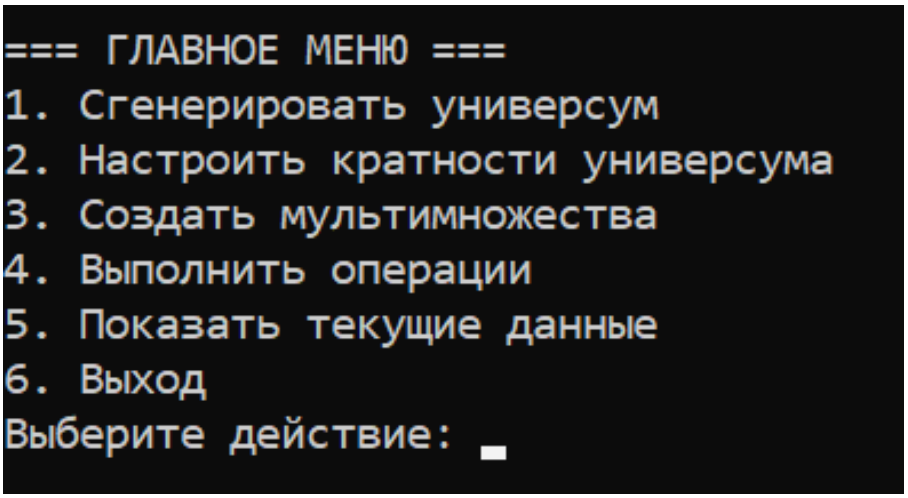


Рис. 1: Главное меню.

Пока у нас не сгенерирован универсум, попробуем выбрать действия 2, 3, 4 и 5.

Окажется, что универсум пустой по умолчанию и все операции будут проводиться для пустого универсума. Для большей прозрачности для пользователя стоит добавить предупреждающее сообщение. Все действия приведены на рисунках 2, 3, 4, 5.



Рис. 2: Действие 2.

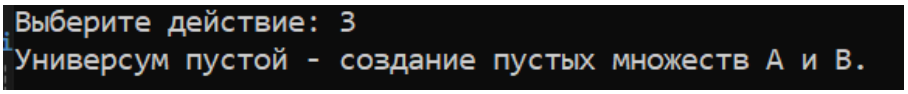


Рис. 3: Действие 3.

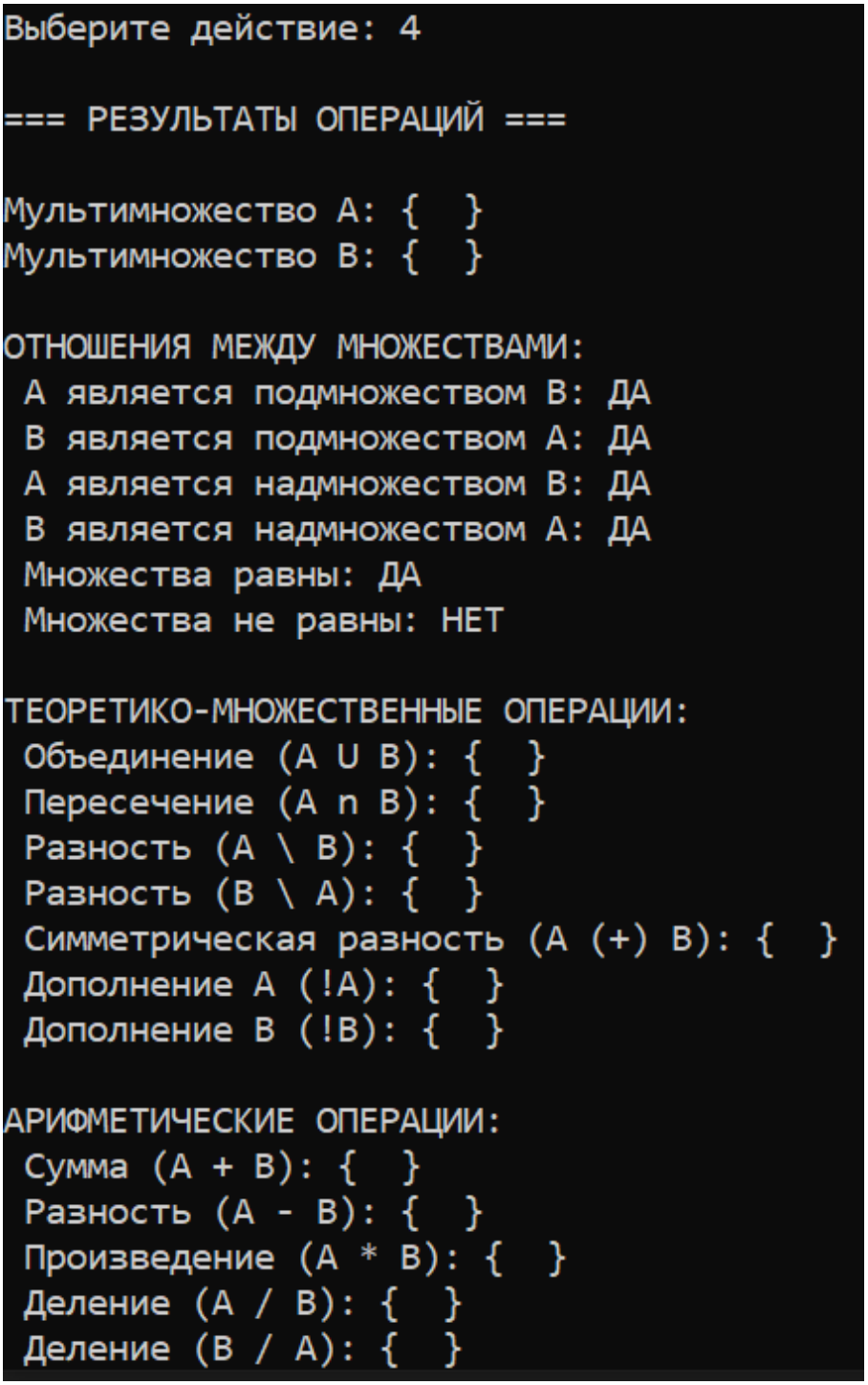


Рис. 4: Действие 4.

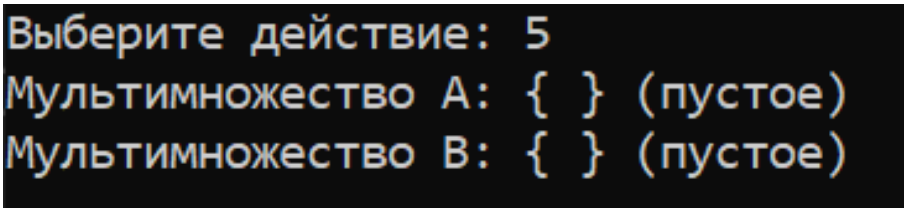


Рис. 5: Действие 5.

Попробуем заполнить универсум: сразу же попробуем ввести неправильное значение. Результат приведен на рис.6. Программа выдает ошибку и сообщение, в том числе просьбу повторить ввод.

```
Выберите действие: 1
Введите разрядность кода Грея (0-11): 12
Ошибка! Введите число от 0 до 11: Введите разрядность кода Грея (0-11):
```

Рис. 6: Некорректная разрядность.

Теперь заполним универсум правильно, например с разрядностью 3, для не слишком длинного вывода. Результат приведен на рис.7. При корректном вводе нас встречает настройка кратностей, зададим ее случайно(автоматически). После этого программа выводит универсум: каждый элемент с разрядностью и мощность всего универсума.

```
Ошибка! Введите число от 0 до 11: Введите разрядность кода Грея (0-11): 3

=== НАСТРОЙКА КРАТНОСТЕЙ УНИВЕРСУМА ===
Заполнение кратностей для 8 элементов:
1. Случайные значения (1-10)
2. Вручную
Ваш выбор: 1
Кратности установлены случайным образом!

=== УНИВЕРСУМ (код Грея) ===
000 : 0 (макс. кратность: 6)
001 : 1 (макс. кратность: 3)
011 : 3 (макс. кратность: 6)
010 : 2 (макс. кратность: 6)
110 : 6 (макс. кратность: 6)
111 : 7 (макс. кратность: 6)
101 : 5 (макс. кратность: 2)
100 : 4 (макс. кратность: 7)
Мощность универсума(сумма макс. кратностей): 42
```

Рис. 7: Заполнение универсума.

Попробуем ввести неправильную кратность(0) для элемента универсума. Программа выдает ошибку и просит ввести кратность еще раз. Это отображено на рис.8. После ввода кратностей универсум обновился.

```
=== НАСТРОЙКА КРАТНОСТЕЙ УНИВЕРСУМА ===
Заполнение кратностей для 8 элементов:
1. Случайные значения (1-10)
2. Вручную
Ваш выбор: 2
Кратность для элемента 0: 0
Ошибка! Введите число от 1 до 100000: Кратность для элемента 0: 11
Кратность для элемента 1:
```

Рис. 8: Неверная кратность элемента универсума.

Попробуем создать мультимножества. Выберем автоматический способ заполнения множеств. Задаем разрядности 0 - создаются пустые множества(в операциях обрабатываются корректно). Создание и результат приведены на рис.9, рис.10.

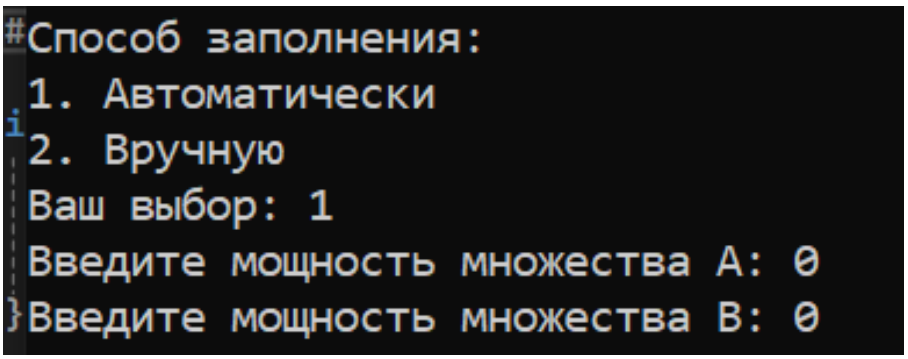


Рис. 9: Задание разрядностей 0.

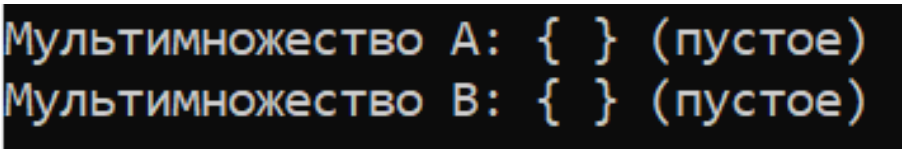


Рис. 10: Создание пустых множеств.

Теперь зададим множества вручную. Программа также позволяет задать пустое множество. Затем попробуем ввести некорректный элемент - программа выдаст ошибку и попросит ввести элемент заново. Действия приведены на рис.11. На рис. 12 приведены итоговые множества с помощью действия 5 - показать текущие данные.

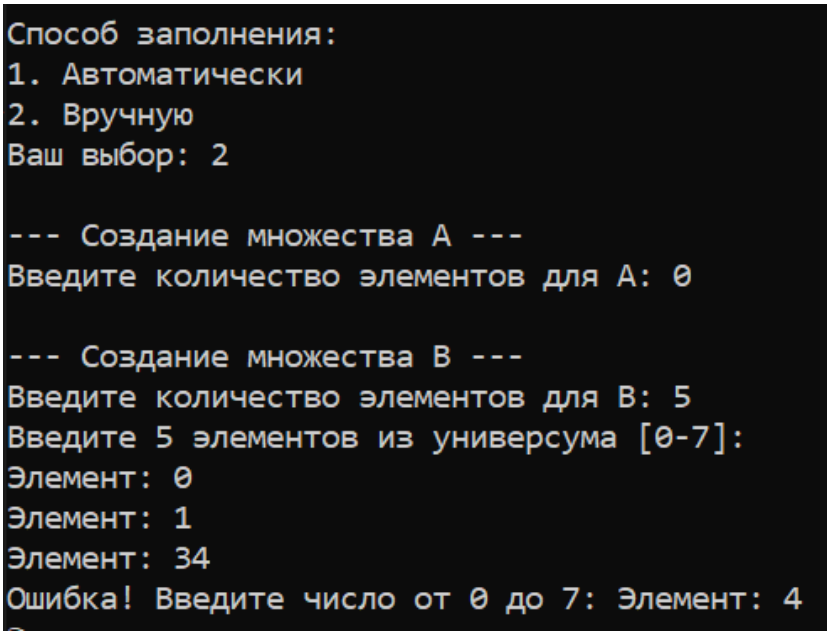


Рис. 11: Задание множеств вручную.

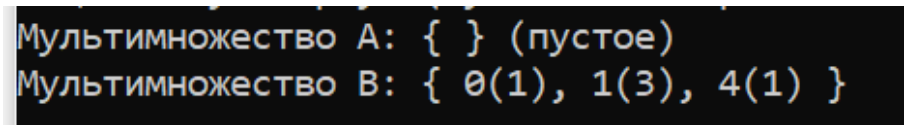


Рис. 12: Итоговые мультимножества.

Выполним операции для данных мультимножеств. Результаты представлены на 13.

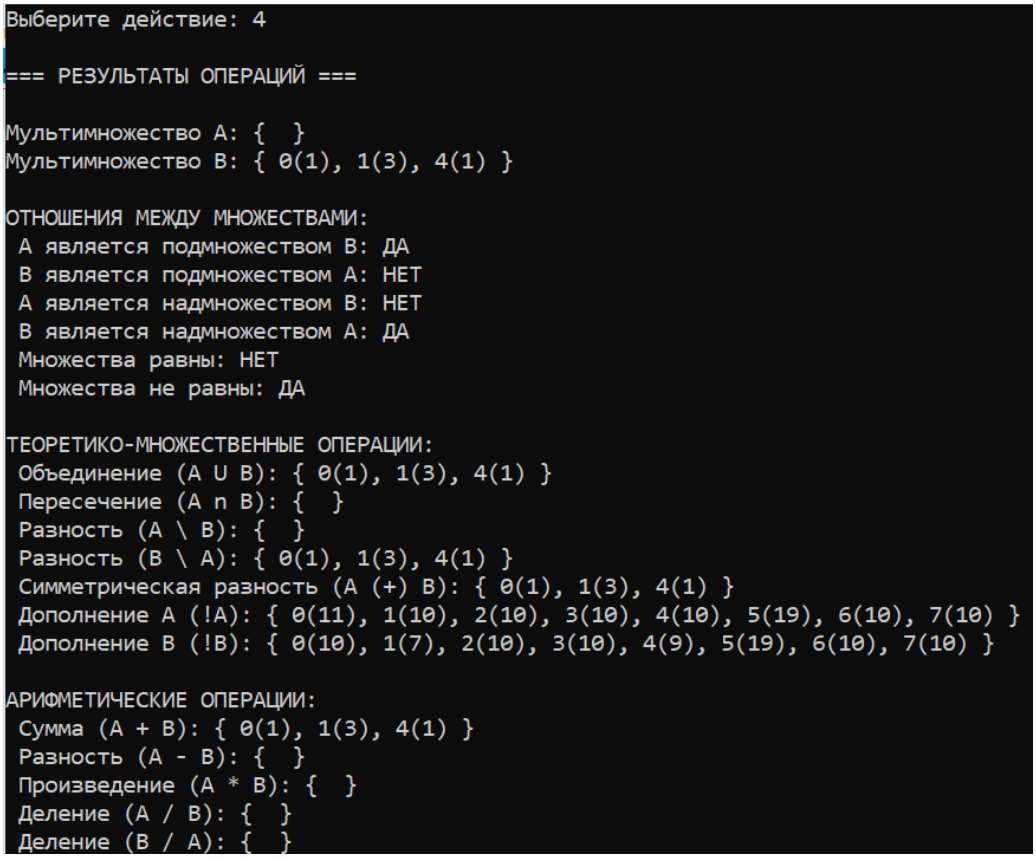


Рис. 13: Результаты операций.

Проверим также команду 6 - выход из программы. Результаты приведены на 14. Команла работает корректно.

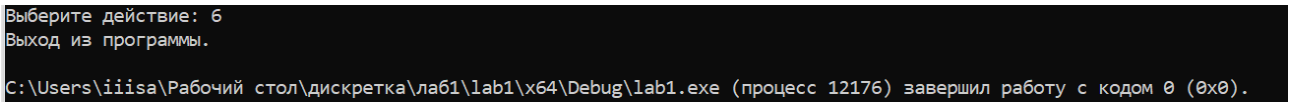


Рис. 14: Выход из программы.

Отдельно хочется отметить обработку случая пустого универсума. Если введем пустой универсум получим следующий результат. Он приведен на рис.15. Далее все действия и их результаты будут такими же, как на рисунках 2, 3, 4, 5.

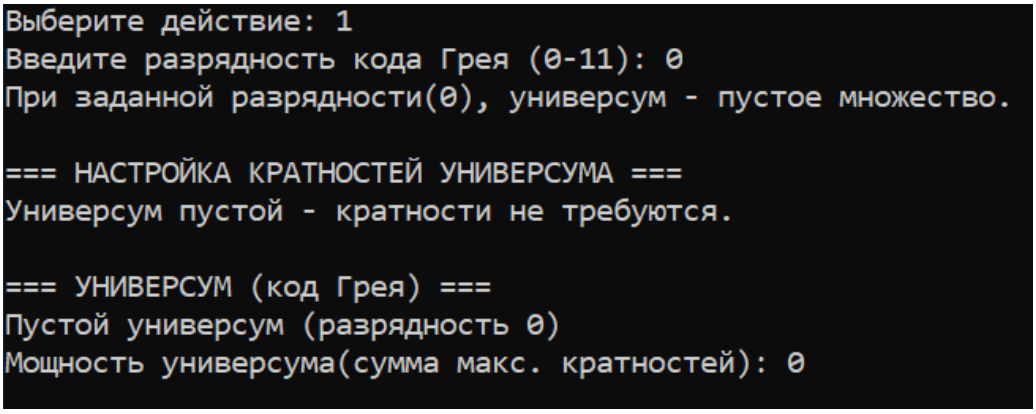


Рис. 15: Обработка пустого универсума.

Заключение

В ходе выполнения лабораторной работы была успешно реализована объектно-ориентированная программа на языке C++ для работы с мультимножествами на основе универсума, созданного с помощью сгенерированного бинарного кода Грея. Программа предоставляет набор возможностей для этого: генерацию кода Грея заданной разрядности, настройку максимальных кратностей элементов универсума, создание двух мультимножеств (автоматически или вручную) с контролем корректности ввода, выполнение теоретико-множественных(пересечение, объединение, дополнение, разность, симметрическая разность) и арифметических операций(сложение, вычитание, умножение, деление), проверку отношений включения и равенства между множествами, а также консольный интерфейс с защитой от некорректного ввода.

Достоинства реализации: структура с чётким разделением ответственности между классами Multiset и Application, что упрощает чтение кода; проверка отношений между множествами(является ли A подмножеством B и наоборот, равны ли множества); Операция разности реализована на основе операций пересечения и разности, а операция симметрической разности – на основе пересечения, объединения и разности, что больше соответствует математическим определениям и формулам, повышает читаемость кода и минимизирует ошибки, так как все операции строятся на основе базовых.

Недостатки реализации: снижение производительности при работе с очень большими универсумами; отсутствие функций сохранения и загрузки данных.

Для масштабирования проекта могут быть предложены следующие улучшения: увеличение максимальной разрядности кода Грея с помощью коррекции вывода(например, выводить не весь универсум, а несколько первых элементов и несколько последних, использовать типы данных long int/long long int для хранения размера универсума и множеств и хранения кратностей универсума и множеств – для обработки больших разрядностей кода Грея); добавление различных функций в пользовательский интерфейс(например, сохранение и загрузка данных из сторонних файлов, задание множеств разными способами: одно вручную, второе автоматически, создание более чем двух мультимножеств и работа с ними в разных комбинациях); оптимизация использования памяти и времени выполнения программмы для работы с большим количеством данных(например, реализовать на основе шаблона функцию выбора типа данных для хранения кратностей на основе выбранной пользователем разрядности универсума; кэшировать часто используемые результаты – вычислять и сохранять дополнение множества при изменении мультимножества, а не пересчитывать каждый раз и т.д.).

Список литературы

[1] Потёмкин И.С. Функциональные узлы цифровой автоматики / И.С. Потёмкин. – М.: Энергоатомиздат, 1988. – 319 с.

[2] Шапорев С.Д. Дискретная математика : учебное пособие / С.Д. Шапорев. – СПб.: БХВ-Петербург, 2007. – 131 с.

[3] Горбатов В.А. Фундаментальные основы дискретной математики : учебное пособие / В.А. Горбатов. – М.: Наука, 1999. – 271 с.

[4] Хаггарти Р. Дискретная математика для программистов / Р. Хаггарти. – М.: Техносфера, 2005. – 400 с.

[5] Новиков Ф.А. Дискретная математика для программистов : учеб. пособие / Ф.А. Новиков. – СПб.: Питер, 2004. – 301 с.